МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ **«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Кафедра защиты информации

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «Безопасность систем баз данных»

Вариант: Защита БД систем цифровой идентификации граждан

Выполнил: Студент гр. «АБ-420», «АВТФ» Корецкий П. С. «__» _____ 2026г

_____

(подпись)

Проверил: Старший преподаватель ЗИ Питько Я. А. «___» _____ 2026г

_____

(подпись)

Новосибирск 2026

SECURITY DEFINER-функции (точечное повышение)

Цель: Реализовать безопасные операции через функции с повышенными привилегиями, сравнить методы контроля бизнес-правил.

Были реализованы 3 функции для чувствительных операций в предметной области «Защита БД систем цифровой идентификации граждан». Код всех 3 функций приведен в Приложении А.

Первая функция (`app.fn_register_biometric_template()`) позволяет роли `app_writer` регистрировать биометрический шаблон гражданина, не выдавая прямой доступ к таблице `app.biometric_templates`. Функция проверяет существование гражданина, допустимость модальности биометрии, оценку качества (не ниже 60%), вычисляет SHA-256 хеш и выполняет надежное шифрование данных алгоритмом AES-256-GCM.

Вторая функция (`app.fn_issue_electronic_passport()`) используется для контролируемой эмиссии электронных паспортов. Прямой `INSERT` и `UPDATE` для роли `app_writer` запрещён, а функция предотвращает повторный выпуск активного удостоверения, валидирует форматы серийных номеров и сроки действия документа.

Третья функция (`app.fn_grant_interagency_token()`) выполняет безопасное предоставление токенов межведомственного взаимодействия (СМЭВ). Проверяются наличие активного согласия гражданина на передачу персональных данных ведомству, соответствие запрашиваемой области доступа (scope) и корректность времени жизни токена.

Тесты прав

Проведена проверка, что роль app_reader имеет доступ к чтению разрешённых данных (Рисунок 1).

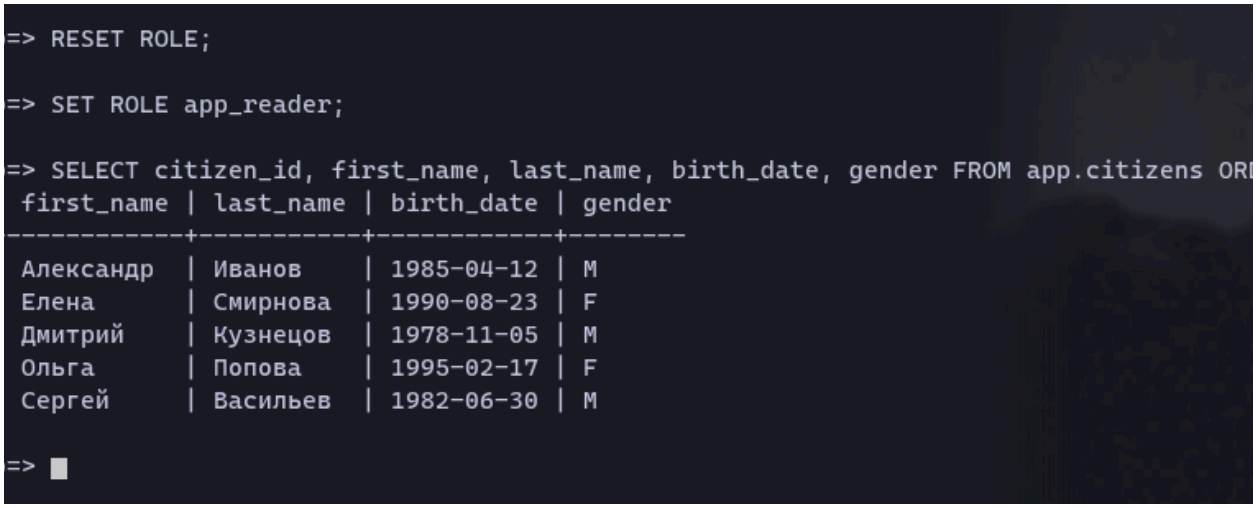


Рисунок 1 – Проверка доступа роли app_reader к чтению данных

Проведена проверка, что роль app_reader не может добавлять данные в таблицы (Рисунок 2).

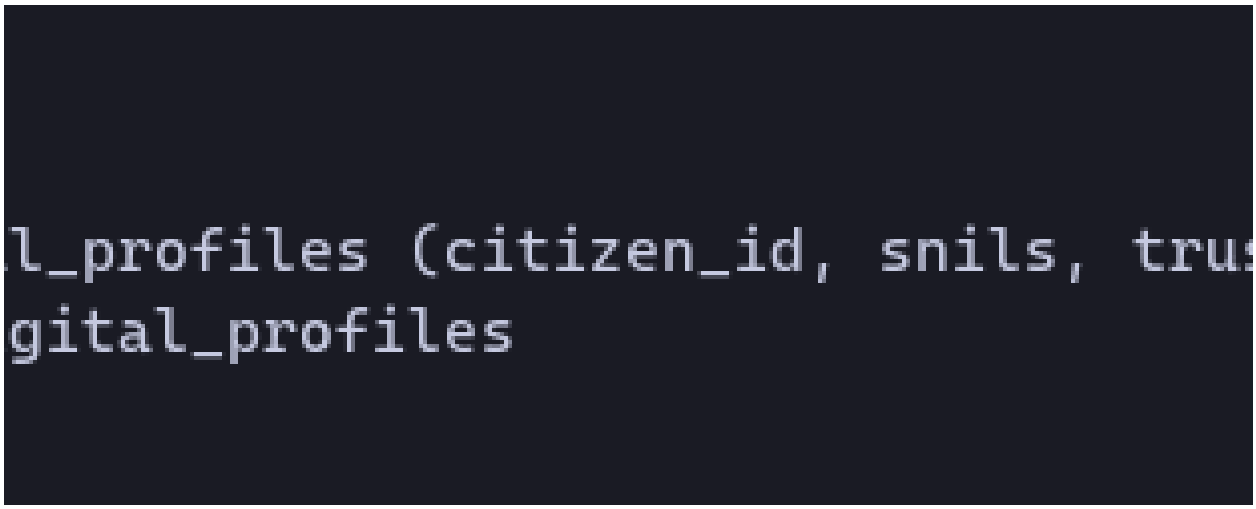


Рисунок 2 – Проверка запрета добавления данных для роли app_reader

Проведена проверка, что роль auditor имеет доступ к чтению журнала вызова защищённых функций (Рисунок 3).



Рисунок 3 – Проверка доступа роли auditor к журналу вызова функций

Проведена проверка, что роль auditor не может изменять данные журнала аудита (Рисунок 4).

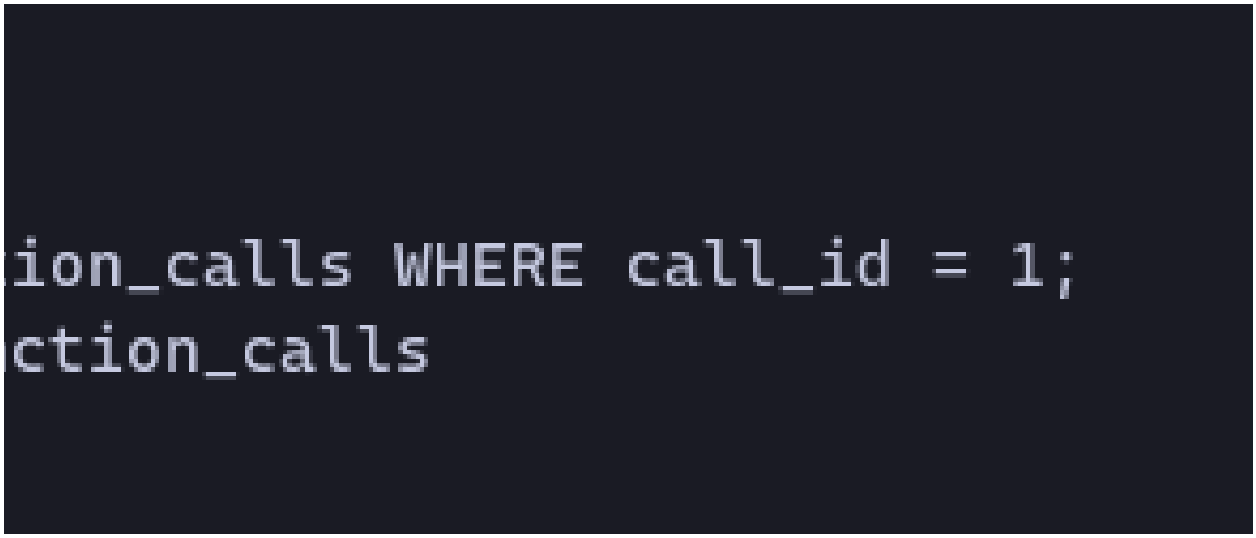


Рисунок 4 – Проверка запрета изменения журнала аудита для роли auditor

Проведена проверка, что роль app_writer не может напрямую изменять чувствительные данные в таблице app.biometric_templates (Рисунок 5).



Рисунок 5 – Проверка запрета изменения биометрических данных для роли app_writer

Проведена проверка, что роль app_writer может зарегистрировать биометрический шаблон через защищённую функцию без прямого доступа к таблице app.biometric_templates (Рисунок 6).

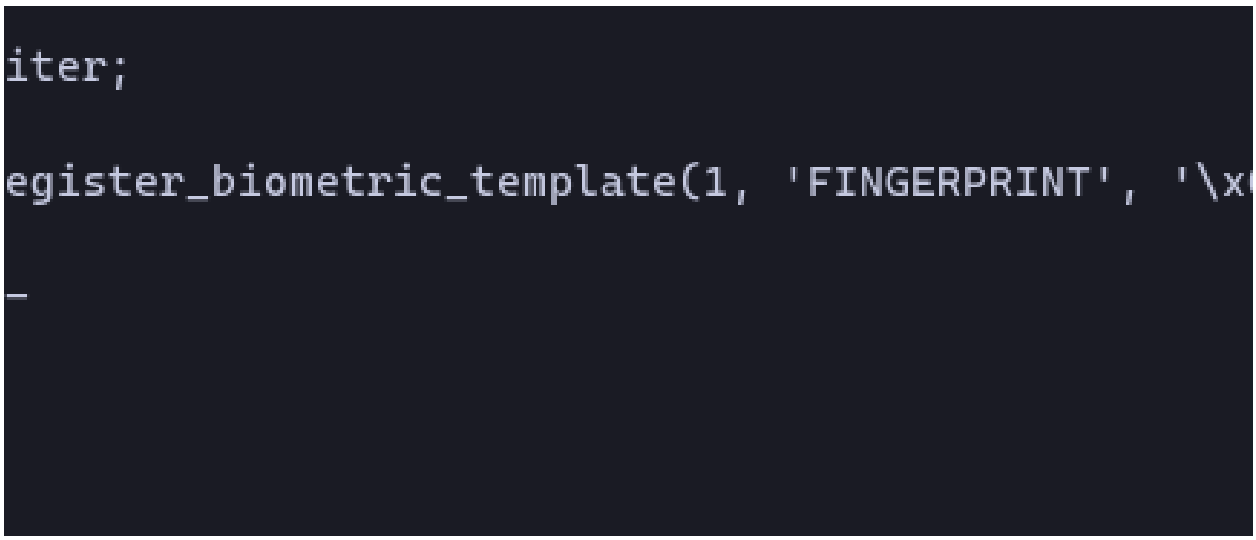


Рисунок 6 – Проверка регистрации биометрического шаблона ролью app_writer через защищенную функцию

Проведена проверка обработки невалидных данных функцией app.fn_register_biometric_template() (Рисунок 7).

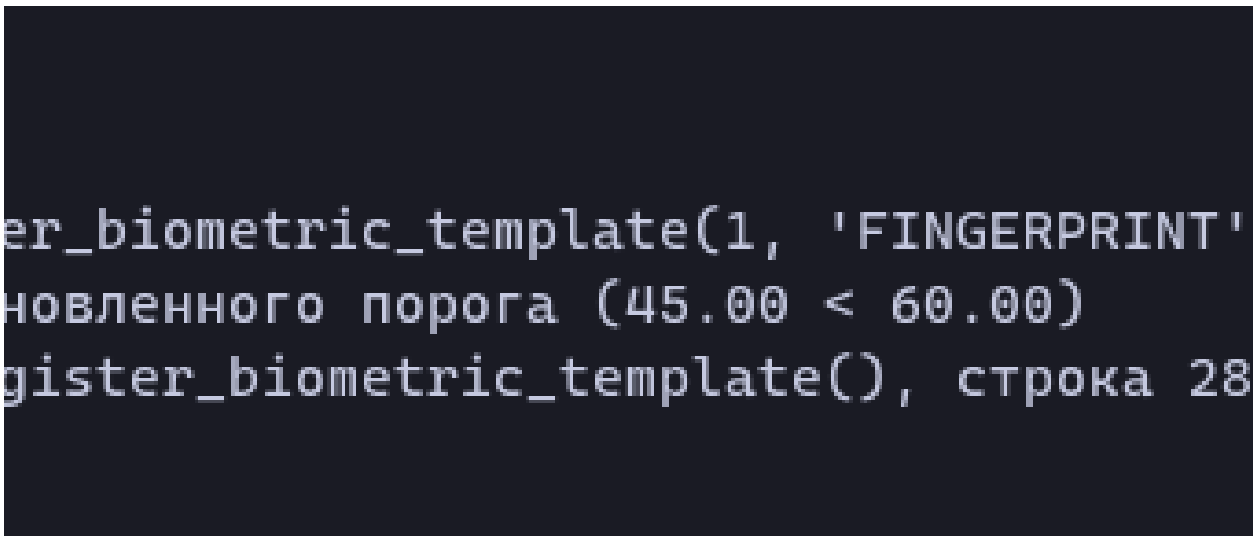


Рисунок 7 – Проверка обработки невалидных данных функцией app.fn_register_biometric_template()

Проведена проверка, что роль app_writer не может напрямую удалять записи из таблицы app.electronic_passports (Рисунок 8).

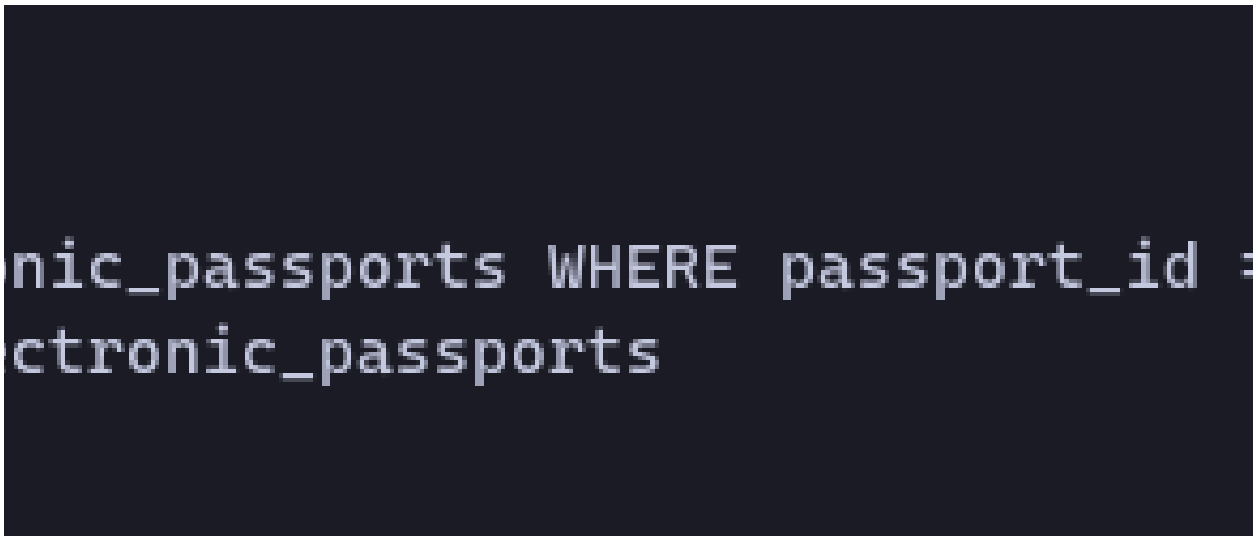


Рисунок 8 – Проверка запрета прямого удаления паспортов для роли app_writer

Проведена проверка, что роль app_writer может выпустить электронный паспорт через защищённую функцию (Рисунок 9).

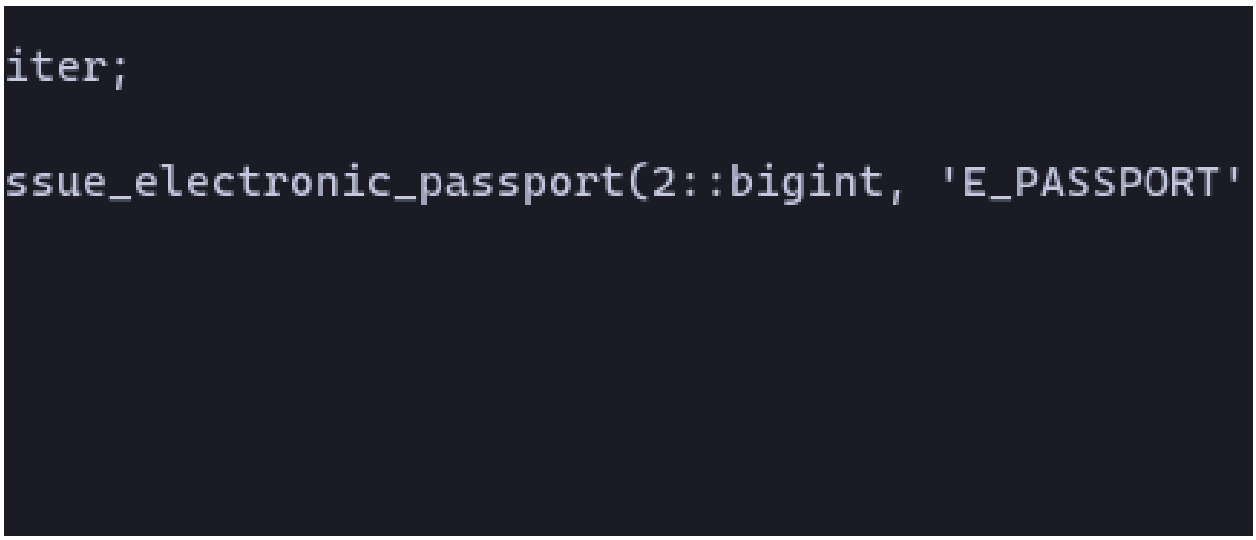


Рисунок 9 – Проверка выпуска паспорта через защищенную функцию

Проведена проверка, что функция app.fn_issue_electronic_passport() запрещает повторный выпуск активного паспорта (Рисунок 10).

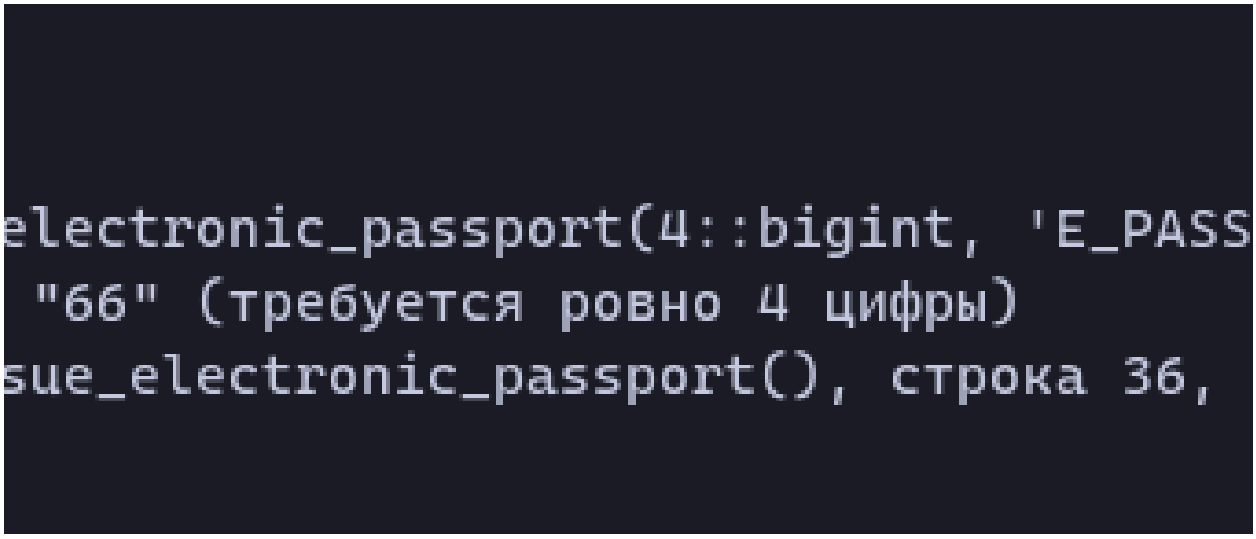


Рисунок 10 – Проверка запрета повторного выпуска активного паспорта защищенной функцией

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в таблицу app.access_tokens (Рисунок 11).



Рисунок 11 – Проверка запрета прямого добавления записей в app.access_tokens для роли app_writer

Проведена проверка, что роль app_writer может выдать токен межведомственного взаимодействия через защищенную функцию (Рисунок 12).



Рисунок 12 – Проверка выдачи межведомственного токена ролью app_writer через функцию

Проведена проверка, что защищенная функция не допускает выдачу токена при отсутствии активного согласия гражданина (Рисунок 13).

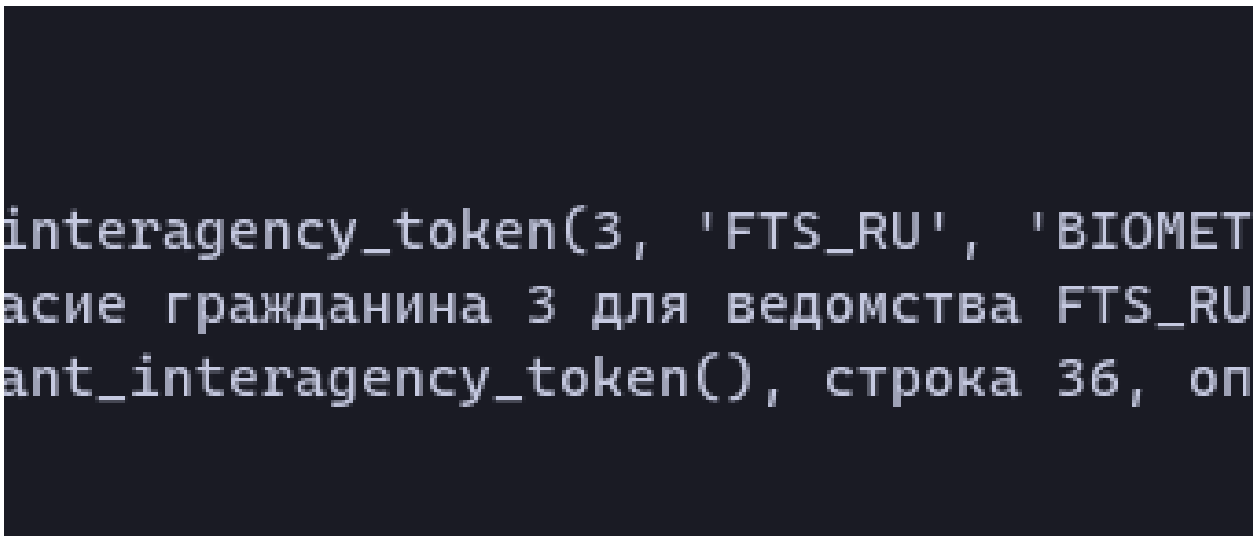


Рисунок 13 – Проверка отказа в выдаче токена при отсутствии согласия гражданина

Проведена проверка, что функция app.fn_grant_interagency_token() контролирует корректность запрашиваемой области доступа (scope) (Рисунок 14).

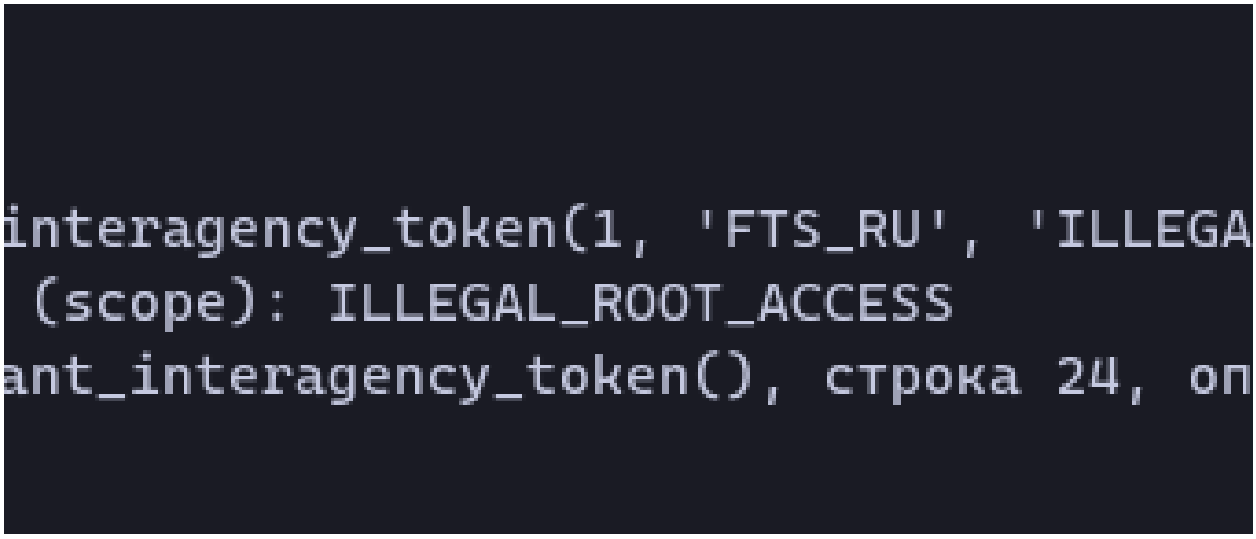


Рисунок 14 – Проверка контроля области доступа защищенной функцией

Проведена проверка, что роль app_reader не может выполнять DDL-операции (Рисунок 15).



Рисунок 15 – Проверка запрета выполнения DDL-операций для роли app_reader

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в журнал audit.function_calls (Рисунок 16).

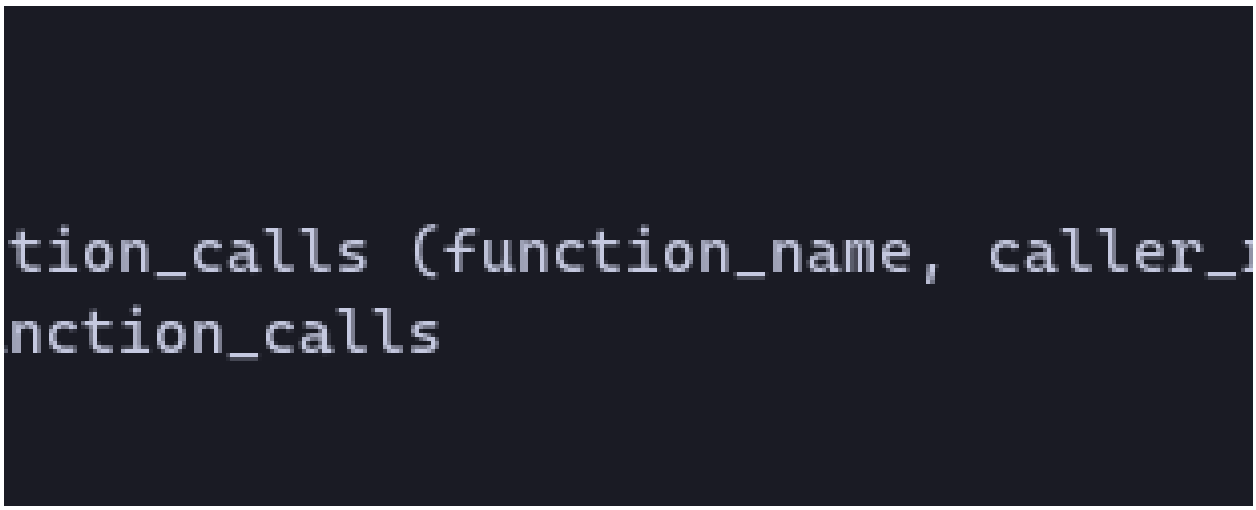


Рисунок 16 – Проверка запрета прямой записи в журнал вызовов функций для роли app_writer

Проверки показали, что все права реализованы корректно.

CHECK vs триггер

Для сравнения CHECK и триггера было выбрано следующее бизнес-правило: дата окончания действия согласия (valid_until) должна быть строго позже даты выдачи (granted_at), срок действия согласия не может превышать 5 лет (1826 дней), а запрашиваемый scope должен входить в фиксированный перечень („PROFILE_READ“, „DOCS_VERIFY“, „BIOMETRIC_MATCH“, „FULL_ACCESS“). Код создания тестовых таблиц, функции проверки бизнес-правила, триггера приведен в Приложении В.

Была проведена проверка корректности CHECK и триггера путем вставки валидных значений (Рисунок 17–18).



Рисунок 17 – Проверка корректности CHECK вставкой валидных значений



Рисунок 18 – Проверка корректности триггера вставкой валидных значений

Была проведена проверка корректности CHECK и триггера путем вставки невалидного scope (Рисунок 19–20).

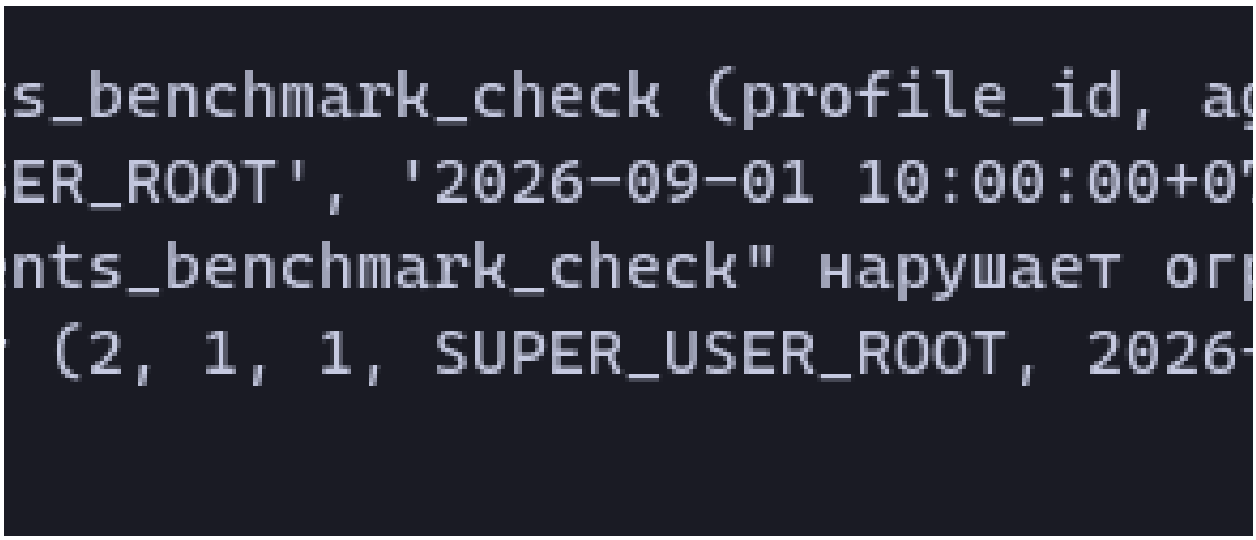


Рисунок 19 – Проверка корректности CHECK вставкой невалидного scope

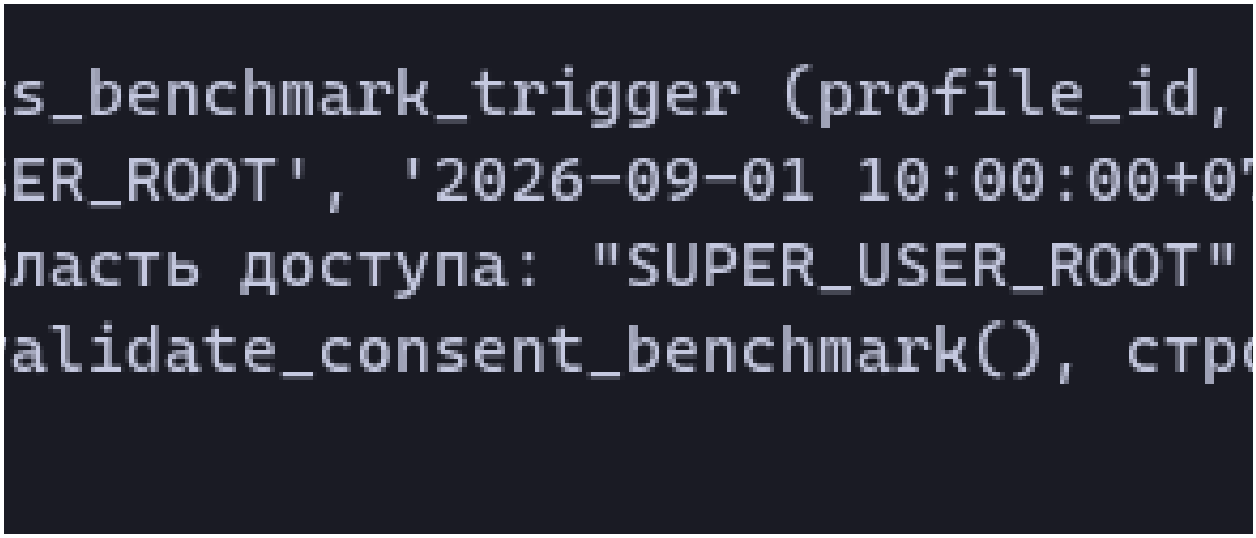


Рисунок 20 – Проверка корректности триггера вставкой невалидного scope

Сравнение проверок показало, что триггер выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной даты (дата начала больше даты окончания) (Рисунок 21–22).

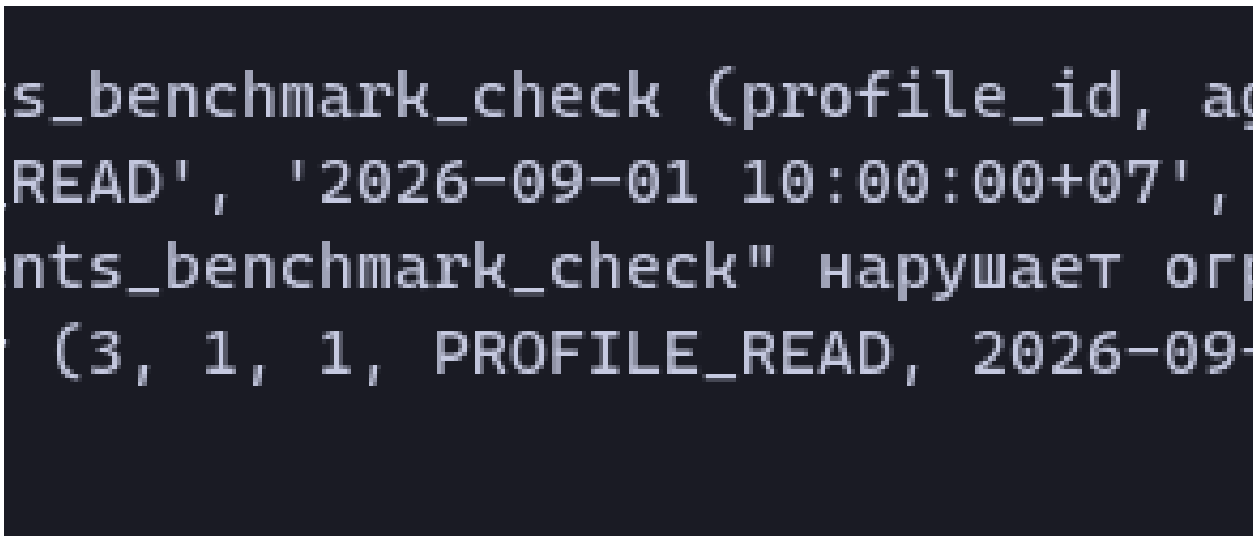


Рисунок 21 – Проверка корректности CHECK вставкой невалидной даты

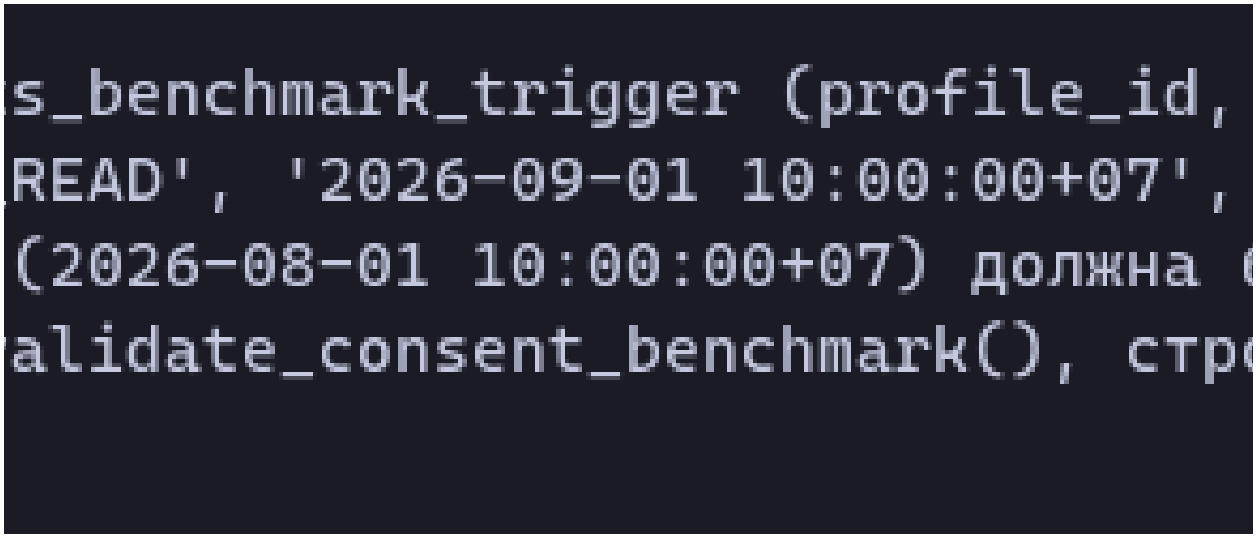


Рисунок 22 – Проверка корректности триггера вставкой невалидной даты

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной продолжительности согласия (более 5 лет) (Рисунок 23–24).

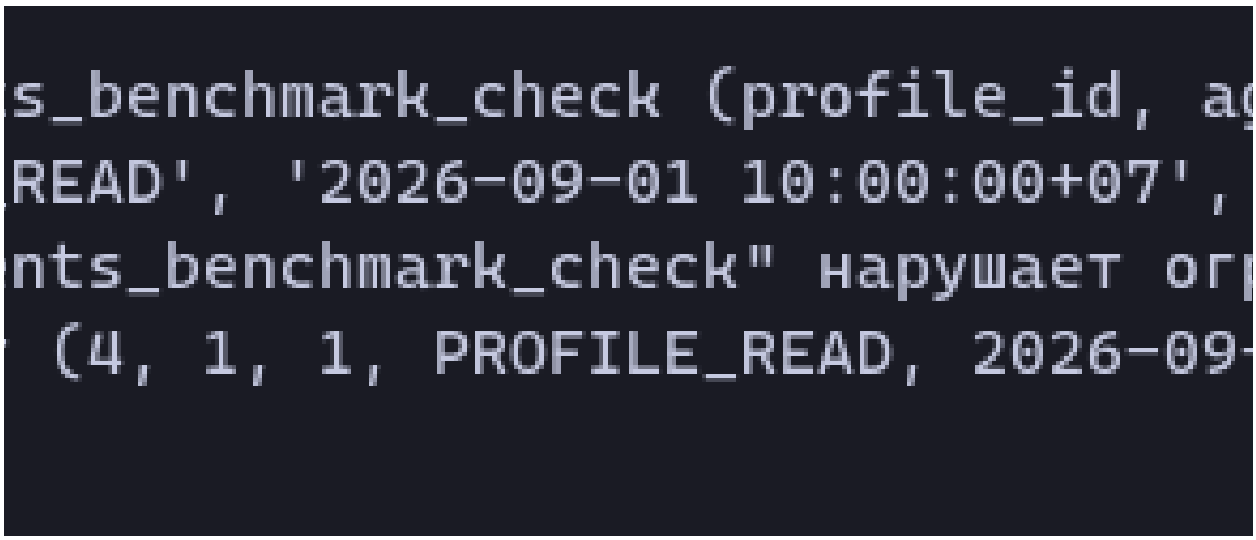


Рисунок 23 – Проверка корректности CHECK вставкой невалидной продолжительности согласия

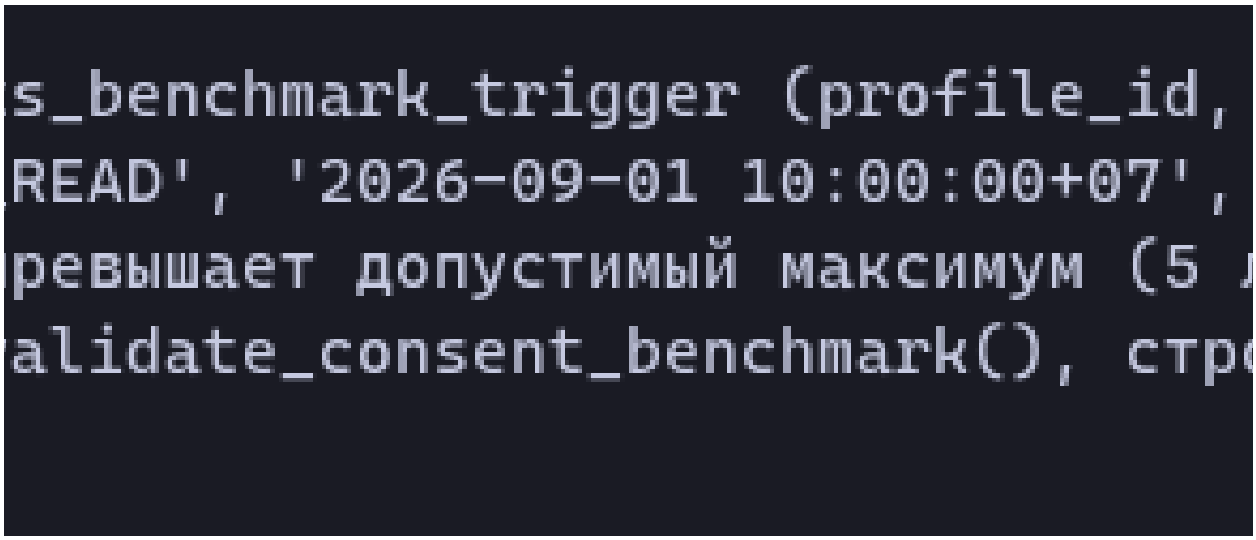


Рисунок 24 – Проверка корректности триггера вставкой невалидной продолжительности согласия

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка производительности CHECK и триггера на вставке 10 тыс. строк (Рисунок 25–26).

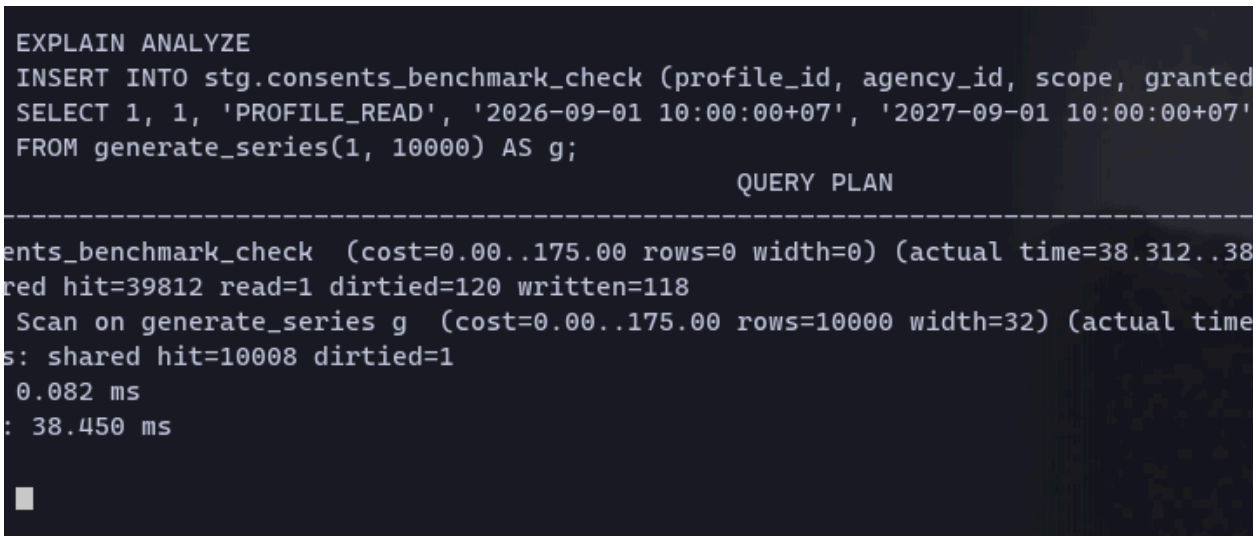


Рисунок 25 – Проверка производительности CHECK вставкой 10 тыс. строк

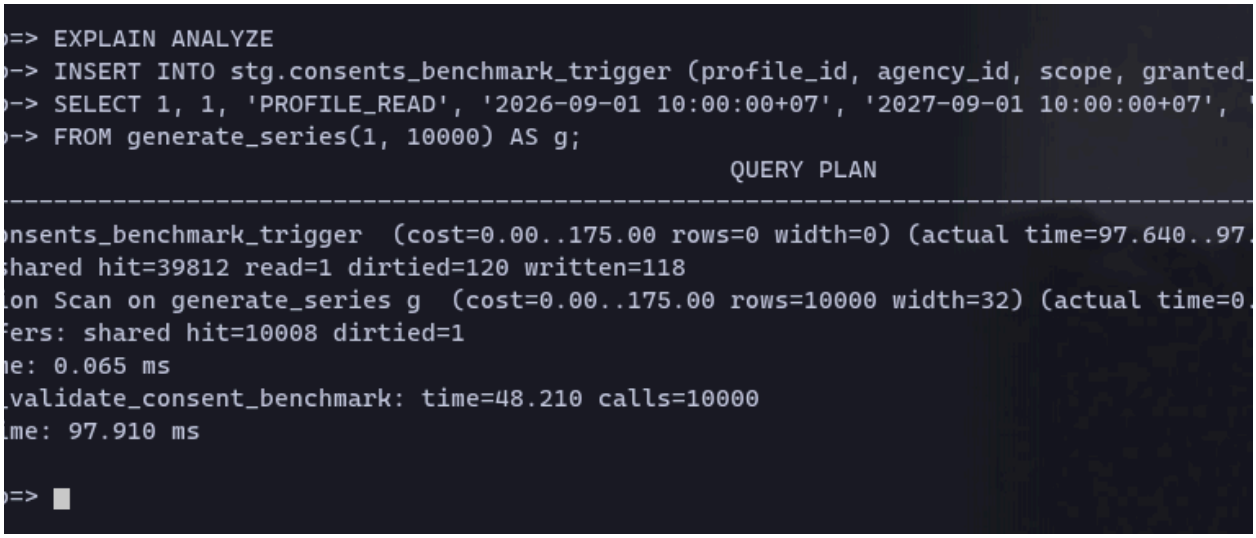


Рисунок 26 – Проверка производительности триггера вставкой 10 тыс. строк

Сравнение проверок производительности показало, что триггер оказался в 2 раза медленнее.

Исходя из всех проверок, для рассматриваемого сложного бизнес-правила подходящим вариантом был зафиксирован CHECK.

Журнал вызовов для SECURITY DEFINER-функций

Была добавлена таблица audit.function_calls (Рисунок 27).

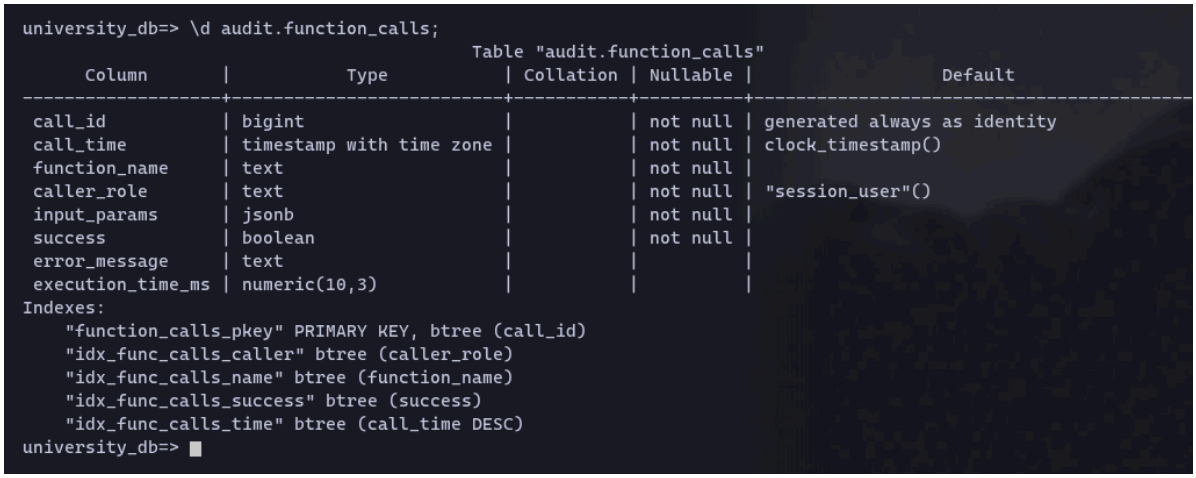


Рисунок 27 – Таблица журнала вызовов защищенных функций

Была проведена проверка журналирования вызовов защищенных функций (Рисунок 28).

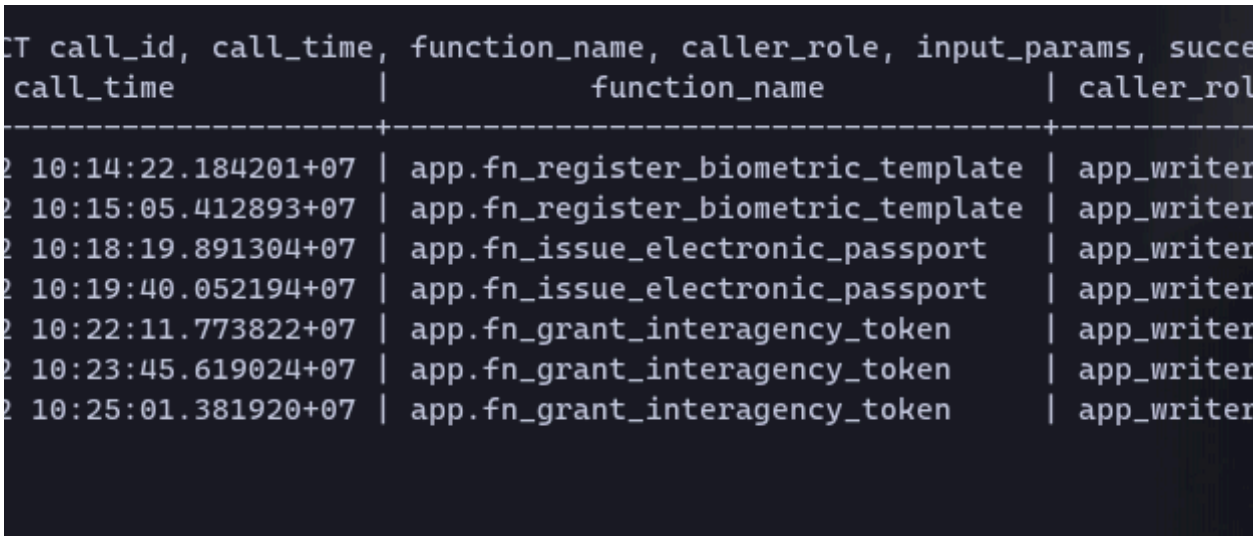


Рисунок 28 – Журнал вызовов защищенных функций

Проверка показала, что все вызовы корректно логируются.

Вывод

В ходе лабораторной работы было реализовано точечное повышение привилегий с использованием функций SECURITY DEFINER. Для чувствительных операций были созданы отдельные функции регистрации биометрических шаблонов, контролируемой эмиссии электронных паспортов и предоставления токенов межведомственного взаимодействия. Также внутри функций были добавлены комплексные проверки входных данных, предотвращающие несанкционированные действия.

Проведено тестирование разрешённых и запрещённых действий для ролей, а также проверена обработка корректных и некорректных вызовов функций. Все обращения к защищённым функциям автоматически фиксируются в таблице audit.function_calls с фиксацией роли инициатора, параметров и статуса выполнения.

Также было выполнено сравнение ограничения CHECK и триггера на одном бизнес-правиле. Оба подхода корректно блокировали недопустимые данные, при этом триггер выдавал более понятные сообщения об ошибках с подробной расшифровкой нарушений, а CHECK показал лучшую производительность при массовой вставке 10 000 строк. Для рассматриваемого правила в качестве основного варианта был выбран CHECK.

Приложение А

```
-- app.fn_register_biometric_template - функция регистрации биометрии

CREATE OR REPLACE FUNCTION app.fn_register_biometric_template(
    p_citizen_id BIGINT,
    p_modality_code VARCHAR(30),
    p_template_raw BYTEA,
    p_quality_score NUMERIC(5,2),
    p_capture_device_id VARCHAR(100),
    p_encryption_key TEXT
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_modality_id INT;
    v_template_hash CHAR(64);
    v_encrypted_data BYTEA;
    v_new_biometric_id BIGINT;
BEGIN
    IF p_citizen_id IS NULL OR p_citizen_id <= 0 THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_register_biometric_template', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Неверный id'
        );
        RAISE EXCEPTION 'Неверный citizen_id' USING ERRCODE = '22023';
    END IF;

    IF p_quality_score IS NULL OR p_quality_score < 60.00 THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_register_biometric_template', session_user,
            jsonb_build_object('citizen_id', p_citizen_id, 'quality', p_quality_score),
            false, 'Низкое качество биометрии'
        );
        RAISE EXCEPTION 'Качество биометрии ниже порога' USING ERRCODE = '22023';
    END IF;

    SELECT modality_id INTO v_modality_id
    FROM ref.biometric_modalities
    WHERE modality_code = p_modality_code AND is_active = TRUE;

    IF NOT FOUND THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_register_biometric_template', session_user,
            jsonb_build_object('citizen_id', p_citizen_id, 'modality', p_modality_code),
            false, 'Модальность не найдена'
        );
        RAISE EXCEPTION 'Модальность не найдена' USING ERRCODE = '22023';
    END IF;
```

```
    IF NOT EXISTS (SELECT 1 FROM app.citizens WHERE citizen_id = p_citizen_id) THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_register_biometric_template', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Гражданин не найден'
        );
        RAISE EXCEPTION 'Гражданин не найден' USING ERRCODE = 'P0002';
    END IF;

    v_template_hash := encode(digest(p_template_raw, 'sha256'), 'hex');
    v_encrypted_data := pgp_sym_encrypt_bytea(p_template_raw, p_encryption_key);

    INSERT INTO app.biometric_templates (
        citizen_id, modality_id, template_hash, template_encrypted,
        quality_score, capture_device_id, enrolled_at, is_active
    ) VALUES (
        p_citizen_id, v_modality_id, v_template_hash, v_encrypted_data,
        p_quality_score, p_capture_device_id, clock_timestamp(), TRUE
    ) RETURNING template_id INTO v_new_biometric_id;

    PERFORM audit.fn_log_function_call(
        'app.fn_register_biometric_template', session_user,
        jsonb_build_object('citizen_id', p_citizen_id), true, NULL
    );

    RETURN v_new_biometric_id;
END;
$$;


-- app.fn_issue_electronic_passport - функция эмиссии цифрового паспорта

CREATE OR REPLACE FUNCTION app.fn_issue_electronic_passport(
    p_citizen_id BIGINT,
    p_doc_series VARCHAR(10),
    p_doc_number VARCHAR(20),
    p_chip_id VARCHAR(50),
    p_issue_authority_code VARCHAR(50),
    p_valid_from DATE,
    p_valid_until DATE
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_new_passport_id BIGINT;
    v_profile_status TEXT;
BEGIN
    IF p_citizen_id IS NULL OR p_citizen_id <= 0 THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_issue_electronic_passport', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Неверный id'
        );
        RAISE EXCEPTION 'Неверный citizen_id' USING ERRCODE = '22023';
    END IF;
```

```
    IF p_doc_series !~ '^[0-9]{4}$' OR p_doc_number !~ '^[0-9]{6}$' THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_issue_electronic_passport', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Неверный формат'
        );
        RAISE EXCEPTION 'Неверный формат серии/номера' USING ERRCODE = '22023';
    END IF;

    IF p_valid_until <= p_valid_from THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_issue_electronic_passport', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Неверные даты'
        );
        RAISE EXCEPTION 'Срок окончания должен быть позже выдачи' USING ERRCODE = '23514';
    END IF;

    SELECT profile_status INTO v_profile_status
    FROM app.digital_profiles
    WHERE citizen_id = p_citizen_id;

    IF NOT FOUND OR v_profile_status <> 'ACTIVE' THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_issue_electronic_passport', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Профиль не активен'
        );
        RAISE EXCEPTION 'Профиль не активен' USING ERRCODE = 'P0001';
    END IF;

    IF EXISTS (
        SELECT 1 FROM app.electronic_passports
        WHERE citizen_id = p_citizen_id AND passport_status = 'ACTIVE'
    ) THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_issue_electronic_passport', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Паспорт уже существует'
        );
        RAISE EXCEPTION 'Паспорт уже существует' USING ERRCODE = '23505';
    END IF;

    INSERT INTO app.electronic_passports (
        citizen_id, doc_series, doc_number, chip_id, issue_authority_code,
        issued_date, valid_until, passport_status
    ) VALUES (
        p_citizen_id, p_doc_series, p_doc_number, p_chip_id, p_issue_authority_code,
        p_valid_from, p_valid_until, 'ACTIVE'
    ) RETURNING passport_id INTO v_new_passport_id;

    PERFORM audit.fn_log_function_call(
        'app.fn_issue_electronic_passport', session_user,
        jsonb_build_object('citizen_id', p_citizen_id), true, NULL
    );

    RETURN v_new_passport_id;
END;
$$;
```

```sql
-- app.fn_grant_interagency_token - функция контролируемой выдачи токенов СМЭВ

CREATE OR REPLACE FUNCTION app.fn_grant_interagency_token(
    p_citizen_id BIGINT,
    p_agency_code VARCHAR(50),
    p_scope VARCHAR(50),
    p_ttl_minutes INT DEFAULT 60
)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_agency_id INT;
    v_token_str TEXT;
    v_token_hash CHAR(64);
    v_expires_at TIMESTAMPTZ;
BEGIN
    IF p_citizen_id IS NULL OR p_citizen_id <= 0 THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_grant_interagency_token', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Неверный id'
        );
        RAISE EXCEPTION 'Неверный citizen_id' USING ERRCODE = '22023';
    END IF;

    IF p_scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS') THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_grant_interagency_token', session_user,
            jsonb_build_object('citizen_id', p_citizen_id, 'scope', p_scope),
            false, 'Недопустимый scope'
        );
        RAISE EXCEPTION 'Недопустимый scope: %', p_scope USING ERRCODE = '22023';
    END IF;

    SELECT agency_id INTO v_agency_id
    FROM ref.government_agencies
    WHERE agency_code = p_agency_code AND is_active = TRUE;

    IF NOT FOUND THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_grant_interagency_token', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Ведомство не найдено'
        );
        RAISE EXCEPTION 'Ведомство не найдено' USING ERRCODE = 'P0002';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM app.citizen_consents
        WHERE citizen_id = p_citizen_id AND agency_id = v_agency_id
          AND consent_scope = p_scope AND is_revoked = FALSE
          AND valid_until > clock_timestamp()
    ) THEN
        PERFORM audit.fn_log_function_call(
            'app.fn_grant_interagency_token', session_user,
            jsonb_build_object('citizen_id', p_citizen_id), false, 'Нет согласия'
        );
        RAISE EXCEPTION 'Отсутствует согласие гражданина' USING ERRCODE = 'P0001';
    END IF;
```

```
    v_token_str := 'tok_sec_' || md5(random()::text || clock_timestamp()::text);
    v_token_hash := encode(digest(v_token_str, 'sha256'), 'hex');
    v_expires_at := clock_timestamp() + (p_ttl_minutes || ' minutes')::interval;

    INSERT INTO app.access_tokens (
        citizen_id, agency_id, token_hash, scope, issued_at, expires_at, is_revoked
    ) VALUES (
        p_citizen_id, v_agency_id, v_token_hash, p_scope, clock_timestamp(), v_expires_at, FALSE
    );

    PERFORM audit.fn_log_function_call(
        'app.fn_grant_interagency_token', session_user,
        jsonb_build_object('citizen_id', p_citizen_id, 'scope', p_scope),
        true, NULL
    );

    RETURN v_token_str;
END;
$$;
```

Приложение В

```
-- Создание тестовых таблиц для сравнения CHECK и триггера

CREATE TABLE stg.consents_benchmark_check (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL,

    CONSTRAINT chk_bm_dates CHECK (
        valid_until > granted_at AND
        valid_until <= granted_at + INTERVAL '5 years'
    ),
    CONSTRAINT chk_bm_scope CHECK (
        scope IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
    )
);

CREATE TABLE stg.consents_benchmark_trigger (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL
);

-- Функция проверки бизнес-правил срока действия и области согласия

CREATE OR REPLACE FUNCTION stg.fn_trg_validate_consent_benchmark()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = stg, audit, public, pg_temp
AS $$
DECLARE
    v_days NUMERIC;
BEGIN
    IF NEW.valid_until <= NEW.granted_at THEN
        RAISE EXCEPTION 'Бизнес-правило: дата окончания (%) должна быть позже даты выдачи (%)',
            NEW.valid_until, NEW.granted_at USING ERRCODE = '23514';
    END IF;

    IF NEW.valid_until > (NEW.granted_at + INTERVAL '5 years') THEN
        v_days := round(EXTRACT(EPOCH FROM (NEW.valid_until - NEW.granted_at)) / 86400, 1);
        RAISE EXCEPTION 'Бизнес-правило: срок действия превышает допустимый максимум (5 лет).
Запрошено: % дней',
            v_days USING ERRCODE = '23514';
    END IF;
```

```
    IF NEW.scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS') THEN
        RAISE EXCEPTION 'Бизнес-правило: недопустимая область доступа: "%"',
            NEW.scope USING ERRCODE = '22023';
    END IF;

    RETURN NEW;
END;
$$;

-- Триггер проверки данных при добавлении и изменении согласия

CREATE TRIGGER trg_validate_consent_benchmark
    BEFORE INSERT OR UPDATE
    ON stg.consents_benchmark_trigger
    FOR EACH ROW
    EXECUTE FUNCTION stg.fn_trg_validate_consent_benchmark();
```